

BUỔI 9 - Hoàn thiện Tính năng Đăng ký Học phần (xuyên 2 Service)

LƯU Ý TRƯỚC KHI BẮT ĐẦU

Điều kiện tiên quyết: đã hoàn thành Buổi 8 - routing, đăng nhập, AuthContext, ProtectedRoute, Role-Based UI hoạt động đúng; RegisterCoursePage hiện mới là khung trang tạm.

Buổi này cần quay lại chỉnh sửa 2 điểm nhỏ ở backend (auth-service, registration-service) trước khi hoàn thiện Frontend - đây là tình huống thực tế rất thường gặp: thiết kế ban đầu luôn có thiếu sót nhỏ lộ ra khi làm tới tính năng cụ thể.

0. Mục tiêu buổi học

Sau buổi học này, sinh viên phải:

- Nhận diện và xử lý một lỗi hỏng thiết kế nhỏ: JWT hiện tại (từ Buổi 4) chỉ mang username/role, chưa mang id của người dùng - trong khi POST /registrations (Buổi 3) lại cần studentId dạng số.
- Bổ sung endpoint GET /registrations/my ở registration-service (đã có trong Blueprint API từ Buổi 1 nhưng chưa triển khai).
- Hoàn thiện trang "Đăng ký học phần": gọi POST /api/registrations qua Gateway, xử lý Toast/Alert cho lỗi nghiệp vụ (hết chỗ, trùng đăng ký) mà không cần phân biệt lỗi đến từ registration-service hay lan truyền từ course-service.
- Hoàn thiện trang "Môn học đã đăng ký": gọi GET /api/registrations/my, cho phép hủy đăng ký qua DELETE /api/registrations/{id}.
- Cập nhật lại danh sách/số chỗ còn lại ngay sau khi đăng ký/hủy thành công, không cần F5 trang.

A. Quay lại auth-service: bổ sung userId vào JWT

A.1. Vì sao cần sửa?

Nhìn lại Registration entity (Buổi 3): trường studentId là kiểu Long. Nhưng JWT sinh ra từ auth-service (Buổi 4) hiện chỉ mang subject = username và claim "role". Frontend không có cách nào biết studentId dạng số của người đang đăng nhập để gửi lên POST /registrations.

Ghi chú:

Đây là ví dụ thực tế rất điển hình trong phát triển phần mềm - một thiết kế ở Buổi 4 tưởng đã đủ, nhưng khi làm tới tính năng cụ thể ở Buổi 9 mới lộ ra thiếu sót. Quay lại sửa 1 service đã "xong" từ trước là chuyện bình thường, không phải làm sai từ đầu.

A.2. Sửa JwtUtil - thêm claim userId

```
// path: auth-  
service/src/main/java/vn/edu/crs/authservice/security/JwtUtil.java  
// purpose: bổ sung thêm số userId vào claim của JWT, giữ nguyên cấu trúc  
còn lại từ Buoi 4  
  
package vn.edu.crs.authservice.security;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.SignatureAlgorithm;  
import io.jsonwebtoken.security.Keys;  
import org.springframework.beans.factory.annotation.Value;  
import org.springframework.stereotype.Component;  
  
import javax.crypto.SecretKey;  
import java.util.Date;  
  
@Component  
public class JwtUtil {  
  
    @Value("${jwt.secret}")  
    private String secret;  
  
    @Value("${jwt.expiration-ms}")  
    private long expirationMs;  
  
    // Thêm thêm số userId, giữ nguyên logic ký token như Buoi 4  
    public String generateToken(Long userId, String username, String role)  
    {  
        SecretKey key = Keys.hmacShaKeyFor(secret.getBytes());  
        Date now = new Date();  
        Date expiry = new Date(now.getTime() + expirationMs);  
  
        return Jwts.builder()  
            .setSubject(username)  
            .claim("userId", userId)  
            .claim("role", role)  
            .setIssuedAt(now)  
            .setExpiration(expiry)  
            .signWith(key, SignatureAlgorithm.HS256)  
            .compact();  
    }  
}
```

A.3. Sửa AuthService và LoginResponseDTO

```
// path: auth-  
service/src/main/java/vn/edu/crs/authservice/dto/LoginResponseDTO.java  
// purpose: bổ sung trường userId để trả về cho Frontend
```

```
package vn.edu.crs.authservice.dto;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class LoginResponseDTO {
    private Long userId;
    private String token;
    private String username;
    private String role;
}
```

```
// path: auth-
// service/src/main/java/vn/edu/crs/authservice/service/AuthService.java
// purpose: cap nhat login() de truyen userId vao generateToken va
// LoginResponseDTO

// Thay dong goi jwtUtil.generateToken(...) cu bang:
String token = jwtUtil.generateToken(user.getId(), user.getUsername(),
user.getRole());
return new LoginResponseDTO(user.getId(), token, user.getUsername(),
user.getRole());
```

A.4. Cập nhật JwtAuthFilter ở course-service và registration-service — đọc thêm userId

```
// path: registration-
// service/src/main/java/vn/edu/crs/registrationservice/security/JwtAuthFilter.java
// (tuong tu ben course-service)
// purpose: bo sung doc claim "userId" tu JWT, gan vao SecurityContext de Controller lay ra
// dung

// Thay doan lay claims va tao Authentication cu bang:
Claims claims = Jwts.parser()
    .verifyWith(key)
    .build()
    .parseSignedClaims(token)
    .getPayload();

String username = claims.getSubject();
String role = claims.get("role", String.class);
Long userId = claims.get("userId", Long.class);

var authToken = new UsernamePasswordAuthenticationToken(
    username, userId, List.of(new SimpleGrantedAuthority("ROLE_" +
```

```
role))
);
```

```
// Lưu ý: trường "credentials" (tham số thứ 2) thường để null, ở đây tạm dùng để chứa
userId
// cho đơn giản - Controller sẽ đọc lại qua Authentication.getCredentials()
SecurityContextHolder.getContext().setAuthentication(authToken);
```

Giải thích lựa chọn kỹ thuật

Thay vì tạo hẳn 1 class *UserPrincipal* riêng (đúng chuẩn Spring Security hơn), buổi này tạm dùng *getCredentials()* để chứa *userId* cho đơn giản.

Ghi chú đây là điểm có thể tinh chỉnh thêm nếu sinh viên làm dự án riêng sau này.

A.5. Cập nhật Frontend - types/auth.ts và AuthContext

```
// path: crs-frontend/src/types/auth.ts
// purpose: bổ sung userId khớp với LoginResponseDTO mới của auth-
service
```

```
export interface LoginRequest {
  username: string;
  password: string;
}
```

```
export interface LoginResponse {
  userId: number;
  token: string;
  username: string;
  role: 'ADMIN' | 'STUDENT';
}
```

```
// path: crs-frontend/src/context/AuthContext.tsx
// purpose: bổ sung id vào AuthUser, giữ nguyên toàn bộ logic còn lại từ Buoi 8
```

```
interface AuthUser {
  id: number;
  username: string;
  role: 'ADMIN' | 'STUDENT';
}
```

```
// Trong ham login(), doi dong tao authUser thanh:
const authUser: AuthUser = { id: data.userId, username: data.username, role: data.role };
```

Test lại auth-service sau khi sửa (qua Gateway): POST <http://localhost:8080/api/auth/login> với student1/student123, kỳ vọng response có thêm trường userId.

```
{
  "userId": 2,
  "token":
"eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJzdHVkZW50MSIsInVzZXJJZCI6Miwicm9sZSI6ImlNUVURFTlQiLCJp
YXQiOiJlE3ODUyOTYwNTYsImV4cCI6MTc4NTM4MjQ1Nn0.hUDX8p5_mXqSTg1qwqusWJYP30y_O9lsKG1UZQK9bt
MKWy-OWMo0QhWlGrHYCD8DqNDLqzDOjL3jYZILTCsAA",
  "username": "student1",
  "role": "STUDENT"
}
```

B. Bổ sung GET /registrations/my ở registration-service

Endpoint này đã có trong Blueprint API viết từ Buổi 1 nhưng chưa từng triển khai - đúng lúc cần dùng ở buổi này.

```
// path: registration-
service/src/main/java/vn/edu/crs/registrationservice/service/RegistrationService.java
// purpose: bổ sung phương thức lấy danh sách đăng ký của 1 sinh viên, giữ nguyên
// các phương thức register/cancel đã có từ Buổi 3

// Thêm phương thức mới trong class RegistrationService:
public List<Registration> getMyRegistrations(Long studentId) {
    return registrationRepository.findByStudentId(studentId);
}
```

```
// path: registration-
service/src/main/java/vn/edu/crs/registrationservice/controller/RegistrationController.java
// purpose: bổ sung endpoint GET /registrations/my, doc studentId từ JWT (qua
Authentication)
// thay vì nhận từ query param - tránh trường hợp 1 sinh viên xem được dữ liệu của sinh
viên khác

import org.springframework.security.core.Authentication;

// Thêm phương thức mới trong class RegistrationController:
@GetMapping("/my")
public List<Registration> getMyRegistrations(Authentication authentication) {
```

```
Long studentId = (Long) authentication.getCredentials();
return registrationService.getMyRegistrations(studentId);
}
```

Điểm bảo mật quan trọng

studentId được lấy từ chính JWT (*authentication.getCredentials()*) chứ không nhận từ query param hay body do Client gửi lên. Nếu để Client tự gửi *studentId* cho endpoint "xem đăng ký của tôi", một sinh viên có thể sửa giá trị đó để xem được dữ liệu đăng ký của sinh viên khác - đây là lỗi bảo mật kinh điển gọi là IDOR (Insecure Direct Object Reference).

Nên nguyên tắc ở đây: dữ liệu định danh người dùng luôn lấy từ token đã xác thực, không bao giờ tin dữ liệu do Client tự khai báo cho việc này.

Cũng nên áp dụng lại nguyên tắc này cho POST /registrations: *studentId* gửi lên từ Frontend vẫn giữ nguyên cách cũ - sinh viên có thể có cách làm chuẩn hơn là backend tự lấy *studentId* từ token thay vì tin Client, có thể coi là bài tập mở rộng sau Buổi 10.

C. Toast - component thông báo dùng chung

```
// path: crs-frontend/src/components/Toast.tsx
// purpose: component thông báo nội (toast) dùng chung cho cả hệ
thống, tu động biến mất sau vài giây
```

```
import { useEffect } from 'react';
```

```
interface ToastProps {
  message: string;
  type: 'success' | 'error';
  onClose: () => void;
}
```

```
export default function Toast({ message, type, onClose }:
ToastProps) {
  useEffect(() => {
    const timer = setTimeout(onClose, 3500);
    return () => clearTimeout(timer);
  }, [onClose]);

  return (
```

```

<div
  style={{
    position: 'fixed',
    bottom: 24,
    right: 24,
    padding: '12px 20px',
    borderRadius: 8,
    color: '#fff',
    backgroundColor: type === 'success' ? '#15803d' : '#b91c1c',
    boxShadow: '0 2px 8px rgba(0,0,0,0.2)',
    zIndex: 1000,
  }}
>
  {message}
  <button
    onClick={onClose}
    style={{ marginLeft: 12, background: 'none', border: 'none',
color: '#fff', cursor: 'pointer' }}
  >
    ×
  </button>
</div>
);
}

```

```

// path: crs-frontend/src/hooks/useToast.ts
// purpose: hook nho giup goi Toast tu bat ky trang nao ma khong can
lap lai state moi lan

import { useState, useCallback } from 'react';

export function useToast() {
  const [toast, setToast] = useState<{ message: string; type:
'success' | 'error' } | null>(null);

  const showToast = useCallback((message: string, type: 'success' |

```

```
'error') => {
    setToast({ message, type });
  }, []);

  const clearToast = useCallback(() => setToast(null), []);

  return { toast, showToast, clearToast };
}
```

D. Hoàn thiện RegisterCoursePage

D.1. API cho Registration

```
// path: crs-frontend/src/api/registrationApi.ts
// purpose: các hàm gọi API liên quan Registration qua Gateway

import axiosClient from './axiosClient';
import type { Registration, RegistrationRequest } from
'../types/registration';

export const registerCourse = (payload: RegistrationRequest) => {
  return axiosClient.post<Registration>('/api/registrations',
payload);
};

export const cancelRegistration = (id: number) => {
  return axiosClient.delete(`/api/registrations/${id}`);
};

export const getMyRegistrations = () => {
  return axiosClient.get<Registration[]>('/api/registrations/my');
};
```

D.2. Bổ sung nút "Đăng ký" vào CourseList

```
// path: crs-frontend/src/components/CourseList.tsx
```

```
// purpose: bo sung prop onRegister (tuy chon) - trang Dang ky hoc
phan se dung prop nay,
// cac trang khac (CoursesPage, AdminCoursesPage) khong truyen vao
nen khong bi anh huong
```

```
import type { Course } from '../types/course';
import type { LoadState } from '../api/useCourses';
```

```
interface CourseListProps {
  courses: Course[];
  state: LoadState;
  errorMessage: string;
  onRetry: () => void;
  onEdit?: (course: Course) => void;
  onDelete?: (course: Course) => void;
  onRegister?: (course: Course) => void;
  registeringId?: number | null; // id mon dang trong qua trinh goi
API dang ky, de disable rieng nut do
}
```

```
export default function CourseList({
  courses, state, errorMessage, onRetry, onEdit, onDelete,
  onRegister, registeringId,
}: CourseListProps) {
  if (state === 'loading') return <p>Dang tai danh sach mon
hoc...</p>;
```

```
  if (state === 'error') {
    return (
      <div style={{ color: '#b91c1c' }}>
        <p>{errorMessage}</p>
        <button onClick={onRetry}>Thu lai</button>
      </div>
    );
  }
```

```
  if (state === 'empty') return <p>Khong tim thay mon hoc nao phu
hop.</p>;
```

```

const showActions = !!onEdit || !!onDelete || !!onRegister;

return (
  <table style={{ width: '100%', borderCollapse: 'collapse' }}>
    <thead>
      <tr style={{ textAlign: 'left', borderBottom: '2px solid #333' }}>
        <th>Ten mon hoc</th>
        <th>So tin chi</th>
        <th>So cho con lai</th>
        {showActions && <th>Thao tac</th>}
      </tr>
    </thead>
    <tbody>
      {courses.map((course) => (
        <tr key={course.id} style={{ borderBottom: '1px solid #eee' }}>
          <td>{course.tenMonHoc}</td>
          <td>{course.soTinChi}</td>
          <td style={{ color: course.soChoConLai === 0 ? '#b91c1c' : 'inherit' }}>
            {course.soChoConLai} / {course.soChoToiDa}
          </td>
          {showActions && (
            <td>
              {onEdit && <button onClick={() =>
onEdit(course)}}>Sua</button>}
              {onDelete && (
                <button onClick={() => onDelete(course)} style={{
marginLeft: 8, color: '#b91c1c' }}>
                  Xoa
                </button>
              )}
            </td>
            {onRegister && (
              <button
                onClick={() => onRegister(course)}
                disabled={course.soChoConLai === 0 ||

```

```

registeringId === course.id}
      >
      {registeringId === course.id
        ? 'Dang dang ky...'
        : course.soChoConLai === 0
        ? 'Het cho'
        : 'Dang ky'}
    </button>
  )}
</td>
)}
</tr>
)}}
</tbody>
</table>
);
}

```

D.3. Trang RegisterCoursePage hoàn chỉnh

```

// path: crs-frontend/src/pages/RegisterCoursePage.tsx
// purpose: trang Dang ky hoc phan hoan chinh - thay the khung trang
tam cua Buoi 8

import { useState } from 'react';
import axios from 'axios';
import { useCourses } from '../api/useCourses';
import { registerCourse } from '../api/registrationApi';
import { useAuth } from '../context/AuthContext';
import { useToast } from '../hooks/useToast';
import SearchBox from '../components/SearchBox';
import CourseList from '../components/CourseList';
import Pagination from '../components/Pagination';
import Toast from '../components/Toast';
import type { Course } from '../types/course';
import type { ApiErrorResponse } from '../types/apiError';

```

```

export default function RegisterCoursePage() {
  const [keyword, setKeyword] = useState('');
  const [page, setPage] = useState(0);
  const [registeringId, setRegisteringId] = useState<number |
null>(null);

  const { user } = useAuth();
  const { toast, showToast, clearToast } = useToast();
  const { courses, totalPages, state, errorMessage, refetch } =
useCourses(keyword, page);

  const handleSearch = (newKeyword: string) => {
    setKeyword(newKeyword);
    setPage(0);
  };

  const handleRegister = async (course: Course) => {
    if (!user) return;
    setRegisteringId(course.id);
    try {
      await registerCourse({ studentId: user.id, courseId: course.id
});
      showToast(`Dang ky thanh cong mon "${course.tenMonHoc}"`,
'success');
      refetch(); // tai lai danh sach de cap nhat so cho con lai moi
nhat
    } catch (err) {
      // LUU Y: loi co the den tu registration-service (vi du "da
dang ky roi")
      // hoac lan truyen tu course-service (vi du "het cho") -
Frontend KHONG can
      // phan biet nguon goc, chi can doc dung truong "message"
trong JSON tra ve.
      let message = 'Dang ky khong thanh cong, vui long thu lai.';
      if (axios.isAxiosError<ApiResponse>(err) &&
err.response?.data?.message) {
        message = err.response.data.message;
      }
    }
  };
}

```

```

    }
    showToast(message, 'error');
  } finally {
    setRegisteringId(null);
  }
};

return (
  <div style={{ padding: 24, maxWidth: 800, margin: '0 auto' }}>
    <h1>Dang ky hoc phan</h1>
    <SearchBox onSearch={handleSearch} />
    <div style={{ marginTop: 16 }}>
      <CourseList
        courses={courses}
        state={state}
        errorMessage={errorMessage}
        onRetry={refetch}
        onRegister={handleRegister}
        registeringId={registeringId}
      />
    </div>
    <Pagination currentPage={page} totalPages={totalPages}
    onPageChange={setPage} />

    {toast && <Toast message={toast.message} type={toast.type}
    onClose={clearToast} />}
  </div>
);
}

```

Nhắc lại điểm nhấn kiến trúc

Dù lỗi "hết chỗ" thật ra được course-service phát hiện và trả về (qua registration-service chuyển tiếp lại), đoạn code handleRegister phía trên không hề biết và không cần biết điều đó - nó chỉ đơn giản đọc `err.response.data.message`. Đây chính là lợi ích cụ thể của việc Frontend chỉ giao tiếp qua 1 điểm vào (Gateway) và 1 format lỗi thống nhất.

E. Trang "Môn học đã đăng ký" - MyRegistrationsPage

E.1. Vấn đề cần giải quyết: hiển thị tên môn học

Registration (Buổi 3) chỉ lưu courseId dạng số, không có tên môn học. Trang này cần hiển thị tên môn học cho dễ đọc - Frontend sẽ tự ghép dữ liệu (API composition) bằng cách gọi thêm GET /api/courses/{id} (endpoint public đã có từ Buổi 2) cho từng courseId xuất hiện trong danh sách đăng ký.

Giải thích lựa chọn kỹ thuật

Đây là 1 trong 2 cách xử lý phổ biến khi dữ liệu nằm rải rác ở nhiều service - (1) để Backend tự ghép (ví dụ registration-service gọi sang course-service rồi trả về dữ liệu đã đầy đủ), hoặc (2) để Frontend tự ghép bằng nhiều lần gọi API. Dự án này chọn cách (2) ở buổi này vì đơn giản hơn cho, tận dụng đúng API public đã có sẵn từ Buổi 2, không cần sửa thêm registration-service.

```
// path: crs-frontend/src/api/courseApi.ts
// purpose: bổ sung hàm lấy 1 môn học theo id, dùng để "ghép" tên môn vào danh sách đăng ký

export const getCourseById = (id: number) => {
  return axiosClient.get<Course>(`/api/courses/${id}`);
};
```

```
// path: crs-frontend/src/pages/MyRegistrationsPage.tsx
// purpose: trang "Môn học đã đăng ký" - lấy danh sách đăng ký rồi
// tu ghép thêm tên môn học,
// cho phép hủy đăng ký

import { useEffect, useState, useCallback } from 'react';
import axios from 'axios';
import { getMyRegistrations, cancelRegistration } from
  '../api/registrationApi';
import { getCourseById } from '../api/courseApi';
import { useToast } from '../hooks/useToast';
import Toast from '../components/Toast';
import type { Registration } from '../types/registration';
import type { Course } from '../types/course';
import type { ApiErrorResponse } from '../types/apiError';

interface RegistrationRow extends Registration {
  courseName: string;
}
```

```
}
```

```
export default function MyRegistrationsPage() {
  const [rows, setRows] = useState<RegistrationRow[]>([]);
  const [loading, setLoading] = useState(true);
  const [loadError, setLoadError] = useState<string | null>(null);
  const [cancellingId, setCancellingId] = useState<number |
null>(null);

  const { toast, showToast, clearToast } = useToast();

  const loadData = useCallback(async () => {
    setLoading(true);
    setLoadError(null);
    try {
      const res = await getMyRegistrations();
      const activeRegistrations = res.data.filter((r) => r.trangThai
=== 'DA_DANG_KY');

      // Ghep ten mon hoc cho tung dong - goi song song bang
Promise.all cho nhanh
      const enriched = await Promise.all(
        activeRegistrations.map(async (reg) => {
          try {
            const courseRes = await getCourseById(reg.courseId);
            return { ...reg, courseName: (courseRes.data as
Course).tenMonHoc };
          } catch {
            // Neu khong lay duoc ten mon (vi du mon da bi Admin
xoa), van hien dong nay
            // voi ten mac dinh, khong lam vo ca trang
            return { ...reg, courseName: `Mon hoc #${reg.courseId}
(khong tim thay thong tin)` };
          }
        })
      );

      setRows(enriched);
    }
  });
}
```

```

    } catch (err) {
      let message = 'Khong tai duoc danh sach dang ky.';
      if (axios.isAxiosError<ApiErrorResponse>(err) &&
err.response?.data?.message) {
        message = err.response.data.message;
      }
      setLoadError(message);
    } finally {
      setLoading(false);
    }
  }, []));

useEffect(() => {
  loadData();
}, [loadData]);

const handleCancel = async (row: RegistrationRow) => {
  if (!window.confirm(`Huy dang ky mon "${row.courseName}"?`))
return;
  setCancellingId(row.id);
  try {
    await cancelRegistration(row.id);
    showToast(`Da huy dang ky mon "${row.courseName}"`,
'success');
    loadData(); // tai lai danh sach
  } catch (err) {
    let message = 'Huy dang ky khong thanh cong.';
    if (axios.isAxiosError<ApiErrorResponse>(err) &&
err.response?.data?.message) {
      message = err.response.data.message;
    }
    showToast(message, 'error');
  } finally {
    setCancellingId(null);
  }
};

return (

```

```

<div style={{ padding: 24, maxWidth: 800, margin: '0 auto' }}>
  <h1>Mon hoc da dang ky</h1>

  {loading && <p>Dang tai...</p>}
  {!loading && loadError && <p style={{ color: '#b91c1c'
}}>{loadError}</p>}}
  {!loading && !loadError && rows.length === 0 && <p>Ban chua
dang ky mon hoc nao.</p>}}

  {!loading && !loadError && rows.length > 0 && (
    <table style={{ width: '100%', borderCollapse: 'collapse'
}}>
      <thead>
        <tr style={{ textAlign: 'left', borderBottom: '2px solid
#333' }}>
          <th>Ten mon hoc</th>
          <th>Ngay dang ky</th>
          <th>Thao tac</th>
        </tr>
      </thead>
      <tbody>
        {rows.map((row) => (
          <tr key={row.id} style={{ borderBottom: '1px solid
#eee' }}>
            <td>{row.courseName}</td>
            <td>{new Date(row.ngayDangKy).toLocaleString('vi-
VN')}</td>
            <td>
              <button onClick={() => handleCancel(row)}
disabled={cancellingId === row.id}>
                {cancellingId === row.id ? 'Dang huy...' : 'Huy
dang ky'}
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  )
}

```

```

    })

    {toast && <Toast message={toast.message} type={toast.type}
onClose={clearToast} />}
    </div>
  );
}

```

F. Cập nhật Route và Navbar

```

// path: crs-frontend/src/App.tsx
// purpose: bổ sung route /my-registrations, giữ nguyên toàn bộ cấu trúc từ Buổi 8

// Thêm import:
import MyRegistrationsPage from './pages/MyRegistrationsPage';

// Thêm route mới bên trong <Routes>, đặt cạnh route /register-course:
<Route
  path="/my-registrations"
  element={
    <ProtectedRoute requiredRole="STUDENT">
      <MyRegistrationsPage />
    </ProtectedRoute>
  }
/>

```

```

// path: crs-frontend/src/components/Navbar.tsx
// purpose: bổ sung link "Mon hoc da dang ky" cho STUDENT, giữ nguyên phần còn lại từ Buổi 8

// Thay đoạn hiển thị menu STUDENT cũ bằng:
{isAuthenticated && user?.role === 'STUDENT' && (
  <>
    <Link to="/register-course">Đăng ký học phần</Link>
    <Link to="/my-registrations">Mon hoc da dang ky</Link>
  </>
)}

```

G. Test theo kịch bản đầy đủ

Đăng nhập bằng student1/student123 (nhớ đăng nhập lại để lấy token mới có userId, token cũ từ Buổi 8 sẽ không còn hợp lệ với registration-service sau khi sửa JwtAuthFilter):

#	Thao tác	Kỳ vọng
1	Vào /register-course, bấm "Đăng ký" một môn còn chỗ	Toast xanh "Dang ky thanh cong...", số chỗ còn lại giảm 1 ngay trên bảng
2	Bấm "Đăng ký" lại đúng môn vừa đăng ký	Toast đỏ hiện đúng message "Sinh vien da dang ky mon hoc nay roi" (lỗi từ registration-service)
3	Dùng Postman/Admin đăng ký hết chỗ 1 môn khác (hoặc tạo môn có soChoToiDa=1 rồi đăng ký bằng tài khoản khác trước), sau đó thử đăng ký môn đó bằng student1	Nút hiện sẵn "Het cho" và bị disable; nếu vẫn cố gọi được (ví dụ do race condition), Toast đỏ hiện đúng message "Mon hoc da het cho" (lỗi lan truyền từ course-service, Frontend không cần biết nguồn gốc)
4	Vào /my-registrations	Thấy đúng danh sách các môn đã đăng ký ở bước 1, kèm tên môn học đã được ghép đúng
5	Bấm "Huỷ đăng ký"	Toast xanh xác nhận, dòng biến mất khỏi bảng
6	Quay lại /register-course	Môn vừa huỷ hiện lại số chỗ đã tăng 1, nút "Đăng ký" hoạt động lại bình thường
7	Đăng nhập bằng admin/admin123, thử gõ tay URL /register-course	Tự chuyển hướng về /courses (đúng theo ProtectedRoute requiredRole="STUDENT" từ Buổi 8)

H. Git commit

```
cd crs-microservices

git add auth-service/
git commit -m "feat(auth-service): add userId claim to JWT"

git add registration-service/
git commit -m "feat(registration-service): add GET /registrations/my endpoint"

git add crs-frontend/
git commit -m "feat: student registration flow across services + error handling"

git push
```

Sản phẩm đầu ra cuối buổi

- [] JWT sinh từ auth-service mang thêm userId, cả course-service lẫn registration-service đọc đúng claim này.

- [] GET /registrations/my lấy studentId từ token đã xác thực, không nhận từ Client — đúng nguyên tắc chống IDOR.
- [] Trang "Đăng ký học phần" gọi POST /api/registrations qua Gateway thành công, hiển thị Toast đúng cho cả 2 loại lỗi nghiệp vụ (trùng đăng ký, hết chỗ) mà không cần phân biệt nguồn gốc lỗi.
- [] Trang "Môn học đã đăng ký" hiển thị đúng tên môn học (ghép dữ liệu từ course-service), cho phép huỷ đăng ký qua DELETE /api/registrations/{id}.
- [] Danh sách/số chỗ còn lại tự cập nhật ngay sau khi đăng ký/huỷ thành công.
- [] Git có 3 commit tách riêng cho auth-service, registration-service, crs-frontend.

Lỗi thường gặp & cách xử lý nhanh

Triệu chứng	Nguyên nhân thường gặp	Cách xử lý
authentication.getCredentials() trả về null hoặc lỗi ép kiểu ClassCastException	Đăng nhập bằng token cũ từ trước khi sửa JwtUtil (token cũ không có userId)	Đăng xuất, đăng nhập lại để lấy token mới
GET /registrations/my luôn trả mảng rỗng dù đã đăng ký thành công	studentId gửi khi POST /registrations (lấy từ user.id ở Frontend) không khớp userId trong token do sai đồng bộ giữa A.5 và D.3	Kiểm tra lại AuthContext đã lưu đúng data.userId chưa
Trang "Môn học đã đăng ký" hiện tên môn dạng "không tìm thấy thông tin" dù môn vẫn còn	getCourseById gọi /api/courses/{id} bị chặn ở Gateway (quên đưa GET /api/courses/** vào danh sách public từ Buổi 4)	Kiểm tra lại AuthHeaderFilter (Buổi 4, mục B.3)
Toast không tự biến mất	Quên clearTimeout hoặc gọi showToast liên tục làm reset timer nhiều lần không kiểm soát	Xem lại mục C, đảm bảo useEffect trong Toast có dependency đúng
Bấm "Đăng ký" 2 lần rất nhanh (double click) tạo 2 request	Chưa disable nút ngay khi bắt đầu gọi API	Kiểm tra lại registeringId đã được set trước khi gọi registerCourse trong handleRegister